VisionInspect AI Milestone 2

VISIONINSPECT AI

AI/ML MODULE | MILESTONE 2

**Advanced Anomaly Detection,
Localization, and Calibration**

PaDiM and PatchCore selection, OpenVINO inference, threshold calibration, heatmaps, mask evaluation, and practical runtime evidence

Submission field	Details
Project	VisionInspect AI
Assigned module	Member 4 - AI/ML
Prepared by	Himanshu Sekhar Parhi
Mentor	Sai Jyoteesh
Programme	Infosys Springboard Programme
Document version	Milestone 2 Submission - Version 1.0
Submission date	1 August 2026

Document Control

This report records Milestone 2 of the VisionInspect AI Member 4 module. It documents the transition from the portable baseline to category-specific PaDiM and PatchCore anomaly models, calibrated decisions, defect localization, and practical advanced-runtime evidence.

Submission sequence: Milestone 1 - 25 July 2026; Milestone 2 - 1 August 2026; Milestone 3 - 8 August 2026; Milestone 4 - 15 August 2026. This report is the second practical stage in that weekly progression.

> **Milestone status:** Completed and verified. All 15 categories have recorded advanced-model metrics and registry selections; cable has a committed OpenVINO runtime and spatial calibrator; focused Milestone 2 tests pass.

Control item	Recorded value
Milestone	2 - Advanced Anomaly Detection, Localization, and Calibration
Primary notebooks	03_advanced_anomaly_detectors.ipynb; 05_localization_and_explainability.ipynb; 07_training_benchmarking_and_calibration.ipynb
Selected models	12 PaDiM categories; 3 PatchCore categories
Advanced executable	Cable PatchCore OpenVINO IR with spatial calibrator
Evaluation state	Category metrics, candidate benchmarks, calibrated thresholds, and holdout records stored in metadata
Focused tests	14 passed, 1 skipped in 4.12 seconds
Repository branch	Himanshu-parhi-ai/ml

Milestone Snapshot

Value	Meaning
15	MVTec categories with advanced model records
12	PaDiM registry selections
3	PatchCore registry selections
0.9554	Mean recorded image AUROC
0.9442	Mean recorded image F1
0.9737	Mean recorded pixel AUROC
0.5402	Mean recorded pixel F1
0.9670	Cable OpenVINO spatial-calibrator F1

Table of Contents

List of Figures

1. Executive Summary

Milestone 2 strengthens VisionInspect AI from a portable anomaly baseline into a category-aware advanced inspection system. The implementation uses Anomalib-based PaDiM and PatchCore training, compares model candidates, stores one selected model family and calibrated threshold per product category, and exposes a shared runtime contract containing an image-level anomaly score, pixel anomaly map, binary mask, detection confidence, and fallback status.

The model registry promotes PaDiM for 12 MVTec categories and PatchCore for cable, hazelnut, and screw. Across the 15 recorded advanced-model evaluations, the unweighted mean image AUROC is 0.9554, image F1 is 0.9442, pixel AUROC is 0.9737, and pixel F1 is 0.5402. The difference between high pixel AUROC and moderate pixel F1 is important: models rank suspicious pixels effectively, but exact binary region overlap remains sensitive to thresholding and defect geometry.

Cable provides the practical deployment demonstration because its selected PatchCore model is committed as OpenVINO IR and paired with a portable ExtraTrees spatial calibrator. On the executed bent-wire example, the engine returned Defective with anomaly score 0.808486, 100% detection confidence, a 6.11% defect-area estimate, severity 77.0 (High), Fail decision, and no fallback. The calibrator's five-

fold out-of-fold evaluation produced F1 0.9670 from 150 labelled cable test images.

> **Evidence boundary:** The repository stores model-selection metadata for all categories, but heavy PaDiM/PatchCore `.ckpt` files are intentionally not committed. The report therefore distinguishes recorded training evaluation from currently executable source-control artifacts.

1.1 Key outcomes

- A single registry controls category, model family, artifact paths, and decision thresholds.
- PaDiM and PatchCore candidates are benchmarked using image and pixel metrics before promotion.
- Advanced thresholds use a stratified calibration/holdout protocol rather than arbitrary values.
- Cable OpenVINO inference uses global and spatial anomaly-map evidence through a 72-value calibrator feature vector.
- Heatmaps, masks, overlap metrics, critical-location geometry, and explainability fields are available for QA review.
- Missing or failed advanced artifacts trigger a recorded portable-baseline fallback.
- Tests protect localization, calibration helpers, runtime artifacts, prediction contracts, and fallback behavior.

2. Milestone Scope and Acceptance Criteria

2.1 Objective

The Milestone 2 objective is to train and evaluate stronger one-class anomaly models, choose the most suitable detector per MVTec category, convert useful model outputs into QA evidence, and prepare a runtime path that remains usable when heavyweight training artifacts are unavailable.

2.2 Included work

- PaDiM and PatchCore model construction through Anomalib.
- Category-specific training on `train/good` data.
- Candidate benchmarking using image AUROC/F1 and pixel AUROC/F1.
- Promotion of the strongest recorded candidate to the category registry.
- Stratified threshold calibration and holdout evaluation.
- OpenVINO export and CPU inference for the cable advanced model.
- Spatial calibration from anomaly score and anomaly-map statistics.
- Heatmap, mask, IoU, Dice, coverage, area, center, and critical-zone evidence.

- Explicit advanced-to-portable fallback semantics.

2.3 Excluded or deferred work

- Defect subtype classification and severity policy are documented in Milestone 3.
- Backend orchestration, batch integration, and final end-to-end validation are documented in Milestone 4.
- This milestone does not implement YOLO-style object detection; anomaly maps localize unusual regions.
- Factory camera, PLC, and MES connections remain outside the AI/ML milestone scope.
- Heavy checkpoint distribution and external model storage are deployment concerns, not source-code evidence.

2.4 Acceptance criteria

Criterion	Verification condition	Status
Advanced model definitions	PaDiM and PatchCore construct successfully from the shared trainer	Met
Category selection	Every supported category resolves to one promoted model kind	Met
Evaluation evidence	Image and pixel metrics are stored for all 15 categories	Met
Calibration	Category thresholds and holdout metrics are persisted where calibration was run	Met
Localization	Runtime returns anomaly map, mask, heatmap, and geometry	Met
Practical runtime	Cable advanced OpenVINO inference completes without fallback	Met
Failure behavior	Unavailable advanced artifacts produce explicit fallback metadata	Met
Software validation	Focused advanced/localization/runtime tests pass	Met

3. Advanced Anomaly Detection Concepts

3.1 Why one-class anomaly detection

Industrial defect datasets rarely enumerate every failure that may occur. PaDiM and PatchCore therefore learn normal appearance from defect-free training images and treat deviations at inference time as anomalies. This matches MVTec AD's normal-only training split and allows the detector to identify previously unseen defect patterns without requiring a closed list of defect classes.

> **Important distinction:** Advanced anomaly detection answers whether and where an image differs from learned normal appearance. It does not by itself name the manufacturing defect subtype; that responsibility belongs to the Milestone 3 classifier.

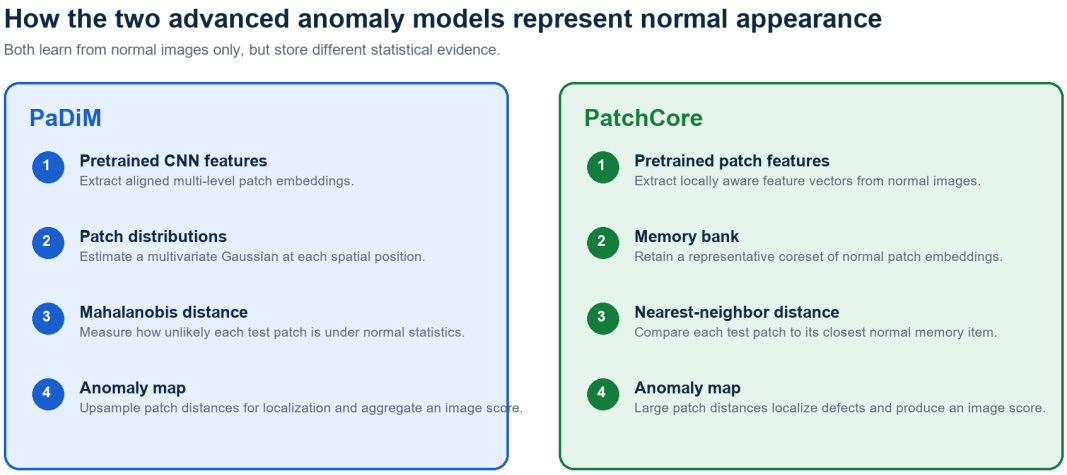

Figure 2. PaDiM and PatchCore representation concepts
Source: Implementation concepts aligned with the referenced PaDiM and PatchCore papers.

3.2 PaDiM role

PaDiM uses pretrained CNN patch embeddings and models the normal feature distribution at each spatial location with a multivariate Gaussian. Test patches receive Mahalanobis-distance anomaly values. The project uses PaDiM as the selected model for 12 categories because it provides strong image-level detection, dense anomaly maps, and comparatively compact normal statistics.

3.3 PatchCore role

PatchCore stores a representative memory bank of normal patch features. At inference, each test patch is compared with nearby normal memory items, and large nearest-neighbor distances indicate anomalies. Candidate benchmarks promoted PatchCore for cable, hazelnut, and screw because it improved their recorded image and/or localization metrics.

Property	PaDiM	PatchCore
Training labels	Normal images only	Normal images only
Normal representation	Per-location Gaussian distributions	Representative patch-memory coreset
Patch score	Mahalanobis distance	Nearest-neighbor feature distance
Outputs	Image score and dense anomaly map	Image score and dense anomaly map
Project selection	12 categories	Cable, hazelnut, screw
Primary limitation	Alignment and covariance assumptions	Larger memory/checkpoint footprint

4. Implementation Architecture

Training and serving are deliberately separated. Training scripts create model and calibration evidence. The runtime reads only the promoted registry entry for the selected category, attempts the available advanced engine, and returns a stable dictionary consumed by later quality-control stages.

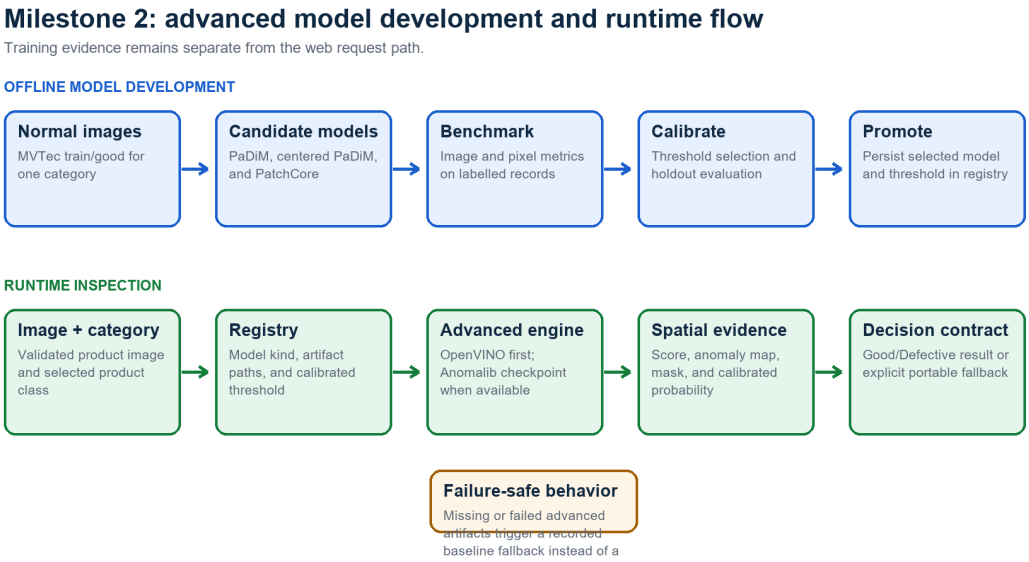

Figure 1. Advanced model development and runtime workflow
Source: Generated from `ml/` and `scripts/` control flow.

4.1 Core implementation modules

Module	Milestone 2 responsibility
ml/anomalib_trainer.py	Construct MVTec datamodule, preprocessing, PaDiM/PatchCore model, and Engine
ml/model_registry.py	Resolve category, model family, artifacts, thresholds, and runtime readiness
ml/padim_detector.py	Load Anomalib/OpenVINO engines, produce score/map/mask, and apply spatial calibrator
ml/inference.py	Choose advanced or portable path and construct the backend-ready inspection contract
ml/heatmap.py	Normalize/colorize maps, overlay masks, and calculate localization metrics
ml/object_preprocessing.py	Foreground extraction, crop, alignment, padding, and contrast utilities

4.2 Runtime result contract

Output field	Meaning
engine	Actual engine, such as `patchcore_openvino`, `padim`, or portable baseline
anomaly_score	Global measure of deviation from normal appearance
decision_threshold	Category-specific threshold stored in the registry/metadata
decision_basis	Score threshold or spatial calibrator
detection_confidence	Confidence attached to Good/Defective detection
anomaly_map	Dense floating-point spatial anomaly evidence
pred_mask	Binary suspicious region used for geometry and heatmap creation
fallback_used / reason	Explicit record when advanced inference cannot run

5. Category Model Registry and Artifact Status

The category registry prevents a cable image from silently using a bottle model. Every request must resolve a supported category, after which the registry supplies model kind, checkpoint path, optional OpenVINO path, optional spatial calibrator, portable profile, classifier artifact, and calibrated thresholds.

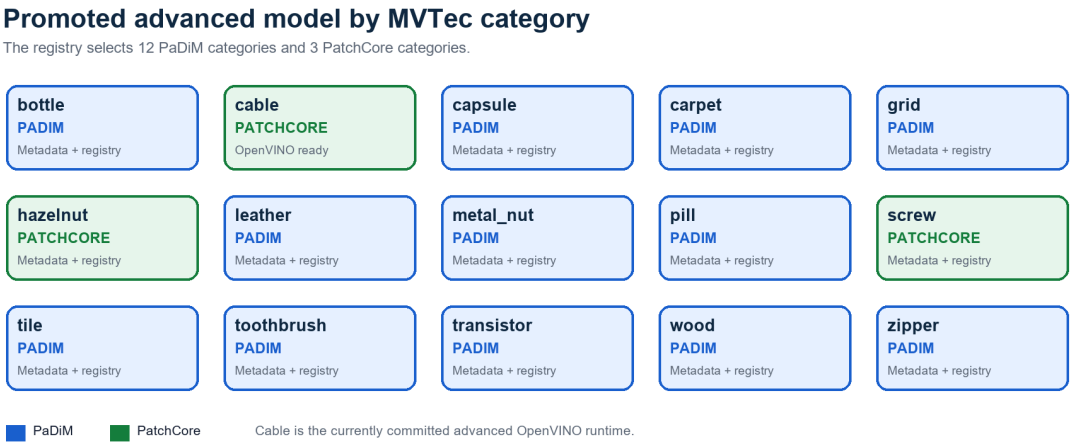

Figure 3. Promoted advanced model by category
Source: Current `models/category_model_registry.json` selections.

5.1 Registry summary

Category	Selected	Threshold	Image F1	Pixel F1	Advanced artifact state
bottle	padim	0.500053	0.9764	0.7141	Checkpoint reference
cable	patchcore	0.513668	0.9274	0.6147	Cable OpenVINO
capsule	padim	0.500863	0.9554	0.4267	Checkpoint reference
carpet	padim	0.616156	0.9724	0.5656	Checkpoint reference
grid	padim	0.487125	0.9298	0.4019	Checkpoint reference
hazelnut	patchcore	0.481546	0.9640	0.6424	Checkpoint reference
leather	padim	0.499993	0.9945	0.3675	Checkpoint reference
metal_nut	padim	0.500032	0.9735	0.7619	Checkpoint reference
pill	padim	0.523497	0.9459	0.5451	Checkpoint reference
screw	patchcore	0.416284	0.9407	0.3717	Checkpoint reference
tile	padim	0.474012	0.9212	0.5253	Checkpoint reference
toothbrush	padim	0.544603	0.9355	0.6088	Checkpoint reference
transistor	padim	0.500030	0.8434	0.6679	Checkpoint reference
wood	padim	0.532627	0.9516	0.4304	Checkpoint reference
zipper	padim	0.500006	0.9316	0.4597	Checkpoint reference

Table 1. Promoted model and threshold by category. `Checkpoint reference` means the registry and evaluation metadata are committed while the heavyweight `.ckpt` file is outside this submission branch.

5.2 Artifact availability

Artifact group	Count	State	Interpretation
Advanced checkpoint references	15	Configured in registry	0 `.ckpt` files committed in this branch
Cable OpenVINO IR	1	Executable	`model.xml` plus 27.45 MB `model.bin`
Cable spatial calibrator	1	Executable	Portable ExtraTrees `.npz` artifact
Advanced metric records	15	Available	Stored in category `model_metadata.json`
Portable normal profiles	15	Executable	Fallback runtime for every category

6. Training, Benchmarking, and Model Promotion

6.1 Reproducible script sequence

Script	Responsibility
train_category_models.py	Train PaDiM and create portable normal profiles
benchmark_category_models.py	Compare current, centered-PaDiM, and PatchCore candidates
promote_category_benchmarks.py	Copy the strongest candidate and update registry metadata
calibrate_category_thresholds.py	Select score thresholds and evaluate holdout behavior
export_openvino.py	Convert available advanced checkpoints to OpenVINO IR

```
python scripts/train_category_models.py --categories cable,hazelnut,screw
python scripts/benchmark_category_models.py --categories cable,hazelnut,screw
python scripts/promote_category_benchmarks.py --categories cable,hazelnut,screw
python scripts/calibrate_category_thresholds.py --categories cable,hazelnut,screw --mode advanced
python scripts/export_openvino.py
```

6.2 Candidate benchmark evidence

Candidate comparisons are stored for cable, grid, hazelnut, and screw. The promotion decision considers image detection and pixel localization together; a model is not selected from a single headline number without its spatial behavior.

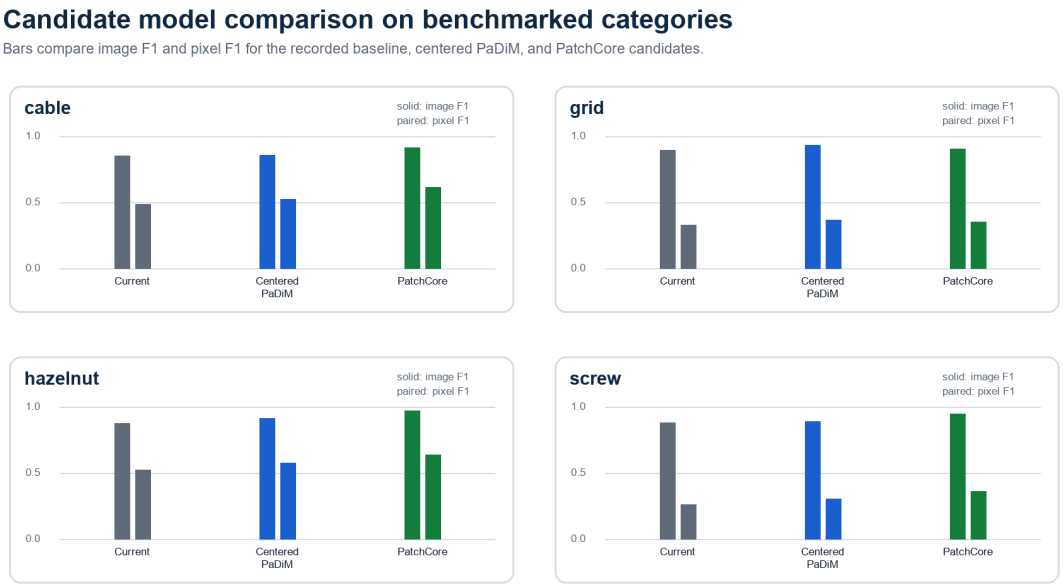

Figure 5. Candidate comparison on benchmarked categories
Source: Current category `benchmark_comparison` metadata.

Category	Candidate	Image AUROC	Image F1	Pixel AUROC	Pixel F1
cable	baseline	0.8774	0.8558	0.9683	0.4860
cable	padim_center_roi	0.9048	0.8617	0.9649	0.5282
cable	patchcore	0.9670	0.9167	0.9816	0.6156
grid	baseline	0.8864	0.9000	0.9583	0.3316
grid	padim_center_roi	0.9607	0.9381	0.9676	0.3710
grid	patchcore	0.9148	0.9057	0.9608	0.3545
hazelnut	baseline	0.8832	0.8767	0.9763	0.5267
hazelnut	padim_center_roi	0.9321	0.9178	0.9773	0.5787
hazelnut	patchcore	0.9961	0.9718	0.9880	0.6403
screw	baseline	0.8307	0.8845	0.9841	0.2641
screw	padim_center_roi	0.8764	0.8923	0.9813	0.3066
screw	patchcore	0.9664	0.9496	0.9918	0.3649

6.3 Promotion findings

- Cable PatchCore increased recorded image F1 from 0.8558 to 0.9167 and pixel F1 from 0.4860 to 0.6156.
- Hazelnut PatchCore increased recorded image F1 from 0.8767 to 0.9718 and pixel F1 from 0.5267 to 0.6403.
- Screw PatchCore increased recorded image F1 from 0.8845 to 0.9496 and pixel F1 from 0.2641 to 0.3649.
- Grid retained PaDiM because centered PaDiM produced the strongest recorded image F1 (0.9381), while PatchCore did not dominate its spatial result.

7. Threshold Calibration and Evaluation Protocol

7.1 Advanced score calibration

For 14 category records, the advanced threshold is selected on a 30% stratified development partition from labelled MVTec test folders and evaluated on the remaining 70% stratified holdout. The calibration script supports F1 or balanced-accuracy objectives. Bottle retains its earlier recorded threshold and model metrics but does not contain the later split-based calibration section, so aggregate holdout values use 14 categories rather than 15.

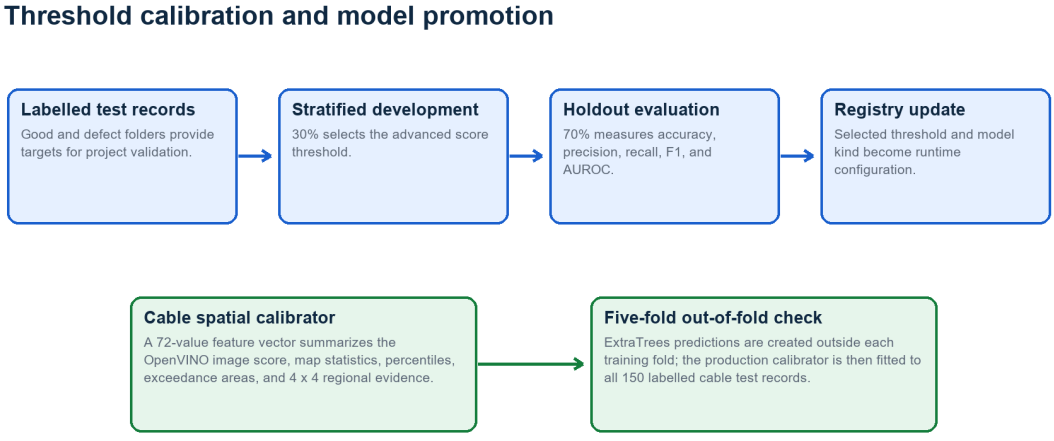

Figure 6. Threshold calibration and model promotion
Source: Implemented by `calibrate_category_thresholds.py` and registry update logic.

Category	Model	Threshold	Accuracy	Precision	Recall	F1	AUROC
cable	patchcore	0.513668	0.8857	0.9643	0.8438	0.9000	0.9703
capsule	padim	0.500863	0.9032	0.9146	0.9740	0.9434	0.8636
carpet	padim	0.616156	0.8902	1.0000	0.8548	0.9217	0.9927
grid	padim	0.487125	0.8909	0.8864	0.9750	0.9286	0.9550
hazelnut	patchcore	0.481546	0.9481	0.9412	0.9796	0.9600	0.9964
leather	padim	0.499993	1.0000	1.0000	1.0000	1.0000	1.0000
metal_nut	padim	0.500032	0.9506	0.9429	1.0000	0.9706	0.9758
pill	padim	0.523497	0.8718	0.9118	0.9394	0.9254	0.8535
screw	patchcore	0.416284	0.9018	0.9000	0.9759	0.9364	0.9713
tile	padim	0.474012	0.8537	0.8615	0.9492	0.9032	0.9528
toothbrush	padim	0.544603	0.8667	0.8696	0.9524	0.9091	0.8413
transistor	padim	0.500030	0.8714	0.7879	0.9286	0.8525	0.9541
wood	padim	0.532627	0.9107	0.9318	0.9535	0.9425	0.9821
zipper	padim	0.500006	0.8774	0.9494	0.8929	0.9202	0.8620

Holdout aggregate: Across the 14 split-calibrated categories, mean holdout accuracy is 0.9016, mean precision is 0.9187, mean recall is 0.9442, mean F1 is 0.9295, and mean AUROC is 0.9408.

7.2 Cable spatial calibration

The cable OpenVINO runtime adds a second decision layer. A 72-value vector summarizes the global score, anomaly-map moments, ten percentiles, nine threshold-exceedance ratios, and three statistics from each cell of a 4 x 4 spatial grid. An ExtraTrees classifier converts that evidence into a defective probability. Five-fold out-of-fold predictions assess the calibrator before the final portable forest is fitted on all 150 labelled cable test records.

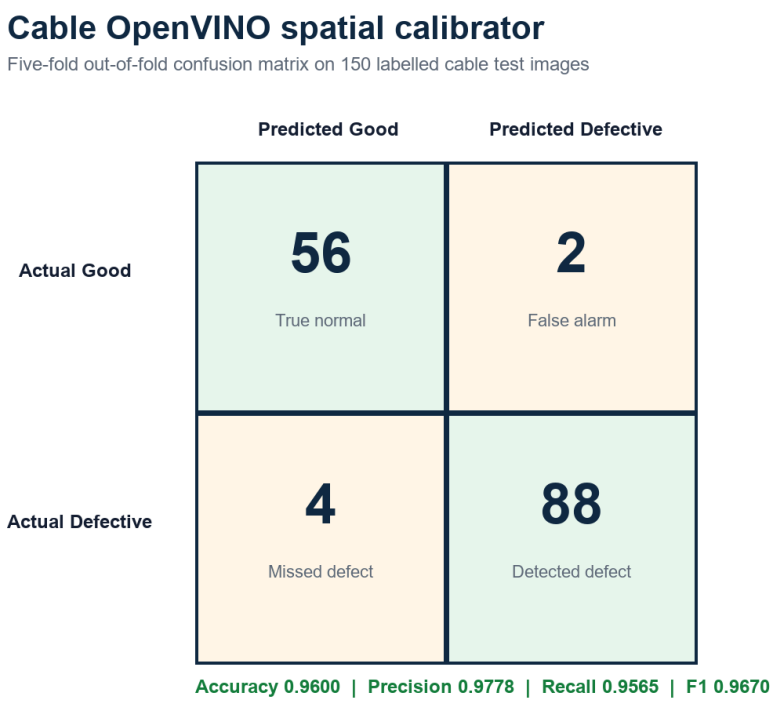

Figure 7. Cable OpenVINO spatial calibrator confusion matrix
Source: Recorded `openvino_spatial_calibration` metadata.

Cable defect subtype	Out-of-fold detection recall
bent_wire	100.00%
cable_swap	100.00%
combined	100.00%
cut_inner_insulation	100.00%
cut_outer_insulation	100.00%
missing_cable	100.00%
missing_wire	80.00%
poke_insulation	80.00%

7.3 Evaluation boundary

These metrics are project validation records derived from labelled MVTec test folders. They are suitable for comparing candidates and calibrating the demonstration system, but they must not be described as an untouched factory acceptance test. A production release requires factory-specific images collected under the target camera, lighting, product orientation, and process conditions.

8. Advanced Model Evaluation Results

8.1 Aggregate view

Aggregate metric	Recorded value
Unweighted mean image AUROC	0.9554
Unweighted mean image F1	0.9442
Unweighted mean pixel AUROC	0.9737
Unweighted mean pixel F1	0.5402
Minimum category image F1	0.8434 (transistor)
Maximum category image F1	0.9945 (leather)

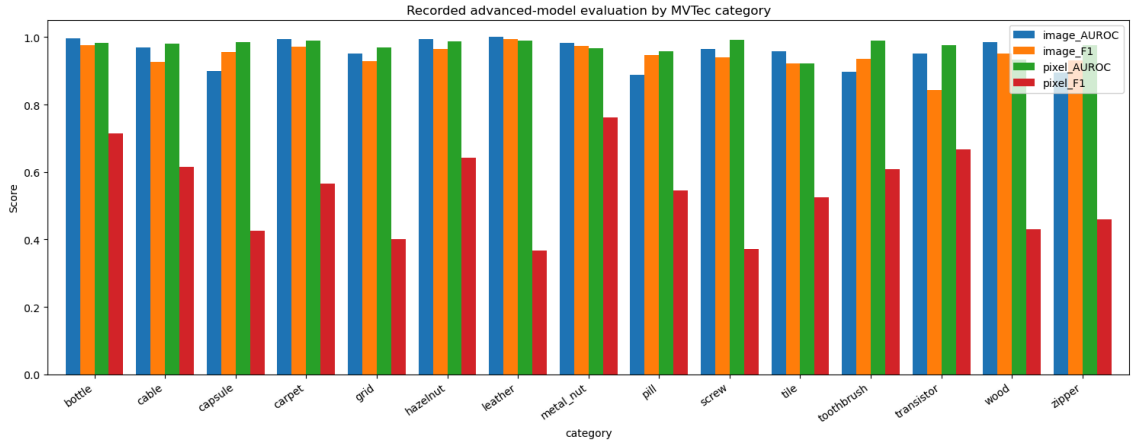

Figure 4. Recorded advanced-model evaluation by category
Source: Executed Notebook 03 using current category model metadata.

8.2 Category metrics

Category	Model	Image AUROC	Image F1	Pixel AUROC	Pixel F1
bottle	padim	0.9968	0.9764	0.9820	0.7141
cable	patchcore	0.9695	0.9274	0.9815	0.6147
capsule	padim	0.8991	0.9554	0.9848	0.4267
carpet	padim	0.9952	0.9724	0.9896	0.5656
grid	padim	0.9524	0.9298	0.9699	0.4019
hazelnut	patchcore	0.9950	0.9640	0.9881	0.6424
leather	padim	1.0000	0.9945	0.9897	0.3675
metal_nut	padim	0.9839	0.9735	0.9679	0.7619
pill	padim	0.8882	0.9459	0.9588	0.5451
screw	patchcore	0.9650	0.9407	0.9918	0.3717
tile	padim	0.9574	0.9212	0.9232	0.5253
toothbrush	padim	0.8972	0.9355	0.9907	0.6088
transistor	padim	0.9525	0.8434	0.9770	0.6679
wood	padim	0.9842	0.9516	0.9343	0.4304
zipper	padim	0.8942	0.9316	0.9754	0.4597

8.3 Interpretation

- All 15 categories have recorded image AUROC above 0.88; the mean is 0.9554.
- Fourteen of 15 categories have recorded image F1 above 0.92; transistor is the lowest at 0.8434.
- Pixel AUROC is strong across categories, indicating useful ranking of suspicious pixels.
- Pixel F1 varies from 0.3675 to 0.7619, so mask thresholding and exact boundary overlap remain the largest localization weakness.
- Metal nut has the strongest recorded pixel F1 (0.7619), while leather and screw show that excellent image detection does not guarantee exact segmentation overlap.
- Model selection is category-specific because texture variation, object alignment, and defect scale affect the two algorithms differently.

9. Practical Advanced Inference Demonstration

9.1 Demonstration setup

Item	Value
Category	cable
Input	Real MVTec cable test image from a defect folder
Selected detector	PatchCore
Execution engine	OpenVINO CPU
Decision basis	Spatial calibrator
Fallback	Not used

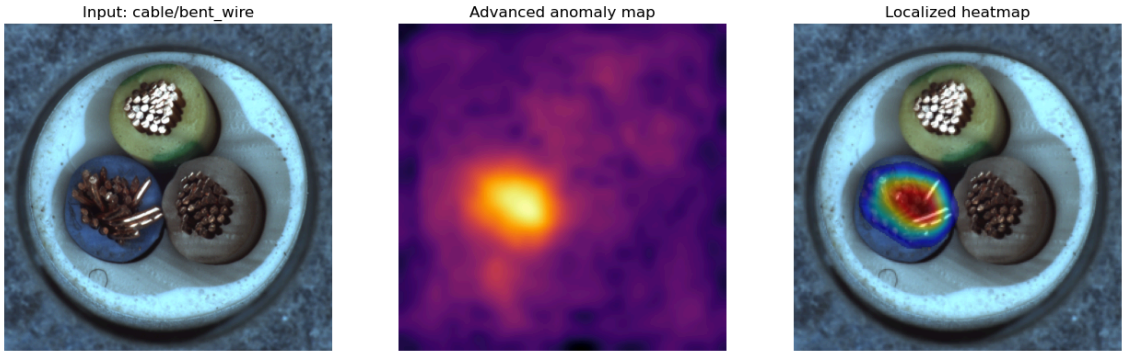

Figure 8. Cable PatchCore OpenVINO practical inference
Source: Executed Notebook 03 on a real MVTec cable image.

9.2 Raw result summary

Output field	Observed value
active_inference_engine	patchcore_openvino
prediction	Defective
defect_type	bent_wire
anomaly_score	0.808486
detection_confidence	1.0000
classification_confidence	0.7967
defect_area_ratio	0.0611 (6.11%)
severity	77.0 - High
quality decision	Fail
model_used	patchcore:v1 (patchcore_openvino)
fallback_used	false

> **What the practical output proves:** The selected category resolved to its promoted engine, the advanced model produced both global and spatial evidence, and the request completed without invoking the portable fallback. The subtype, severity, and QA fields are shown only to demonstrate the downstream interface; their implementation and evaluation belong to Milestone 3.

10. Localization and Explainability

10.1 Shared localization contract

PaDiM, PatchCore OpenVINO, and the portable detector all return an anomaly map and predicted mask through the same interface. The heatmap module normalizes the map for display, overlays it on the source image, overlays binary masks, and compares predicted regions with MVTec ground truth. This common contract lets the frontend and severity logic remain independent of the selected anomaly engine.

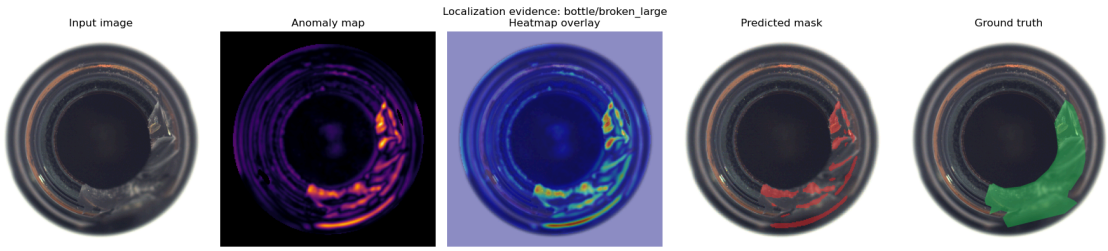

Figure 9. Localization and explainability evidence
Source: Executed Notebook 05 on bottle/broken_large using the shared localization contract.

10.2 Demonstrated overlap and geometry

Measure	Observed value	Interpretation
IoU	0.2977	Intersection divided by union of predicted and ground-truth regions
Dice	0.4588	Twice the overlap divided by combined mask area
Ground-truth coverage	0.3073	Fraction of labelled defect pixels captured
Ground-truth area ratio	0.1089	Labelled defect area relative to image
Predicted area ratio	0.0370	Predicted suspicious area relative to image
Predicted center	x=0.6649, y=0.6854	Normalized defect-mask centroid
Critical location	True	Predicted region intersects configured center/edge critical zones

10.3 Explainability fields used later

Field	QA meaning
Heatmap P95	95th percentile anomaly-map intensity
Defect area	Fraction of predicted-mask pixels
Critical zone	Whether the mask intersects product-configured QA zones
Decision threshold	Exact threshold applied by the active detector/calibrator
Decision basis	Spatial calibrator or direct score threshold
Fallback status	Whether the result came from the requested advanced model
Notes	Human-readable reasons for Good, Defective, Review, or Fail behavior

Localization finding: The demonstrated mask does not cover the entire labelled bottle defect. This is not hidden: the IoU, Dice, and coverage values quantify the gap and justify continued localization refinement.

11. Runtime Portability and Fallback Behavior

11.1 Advanced engine selection

1. Validate the image path and selected MVTec category.
2. Read the category's model family, threshold, and artifact paths from the registry.
3. Use the OpenVINO model when a complete `.xml` and `.bin` pair exists.
4. Otherwise load the PaDiM/PatchCore `.ckpt` through Anomalib when present.
5. Return score, map, mask, confidence, and engine metadata.
6. If advanced inference raises `PadimInferenceError`, run the category portable baseline and set `fallback_used=true` with the reason.

11.2 Source-control packaging

Artifact	Branch state	Runtime/documentation role
Category metadata	Committed	Metrics, thresholds, benchmark comparison, and provenance
Category registry	Committed	Runtime model selection and artifact paths
Cable OpenVINO IR	Committed	Executable CPU advanced detector
Cable calibrator	Committed	Portable spatial decision model
PaDiM/PatchCore checkpoints	Excluded	Heavy training artifacts regenerated or distributed separately
Portable normal profiles	Committed	Fallback coverage for all categories

11.3 Why fallback is not silent

A missing advanced checkpoint must not be reported as if PaDiM or PatchCore ran. The result records the actual engine, fallback flag, and failure reason. This protects demos and deployments from overstating the active model and lets administrators distinguish model-quality errors from packaging or infrastructure errors.


```
{
"active_inference_engine": "portable_baseline",
"fallback_used": true,
"fallback_reason": "PaDiM checkpoint not found: ..."
}
```


12. Testing and Reproducibility

12.1 Focused automated tests

The focused command validates heatmap behavior, mask overlap metrics, k-fold helpers, portable artifact availability, advanced prediction contracts, OpenVINO spatial features, explainability, and explicit fallback semantics. Fourteen tests passed. One controlled skip covers an environment-dependent artifact case and is not a failed assertion.

```
PS C:\VisionInspectAI-team> python -m pytest
tests/test_heatmap.py tests/test_kfold_validation.py
tests/test_portable_runtime.py tests/test_prediction.py -q

...s..........                                          [100%]
14 passed, 1 skipped in 4.12s

Validated: heatmaps, localization metrics, calibration helpers,
portable artifacts, runtime contract, fallback, and explainability.
```

Figure 10. Focused Milestone 2 automated test result
Source: Focused automated test command output.

Test area	Behavior protected
Heatmap normalization	Finite, correctly shaped anomaly visualizations
Zero-map behavior	Good images do not receive a false blue heatmap tint
Mask metrics	IoU, Dice, and overlay behavior
K-fold utilities	Stable threshold and metric aggregation helpers
Portable runtime	Every category has baseline artifacts when advanced files are absent
OpenVINO features	Fixed-length finite spatial feature vector
Prediction contract	Detection confidence remains separate from subtype confidence
Explainability	Decision basis, threshold, and spatial evidence are described
Fallback	Missing advanced artifact is recorded rather than hidden

12.2 Notebook reproduction

```
python -m pip install -r requirements-ai.txt
$env:MVTEC_DATASET_ROOT = 'C:\path\to\mvtec_anomaly_detection'
jupyter nbconvert --to notebook --execute --inplace notebooks/03_advanced_anomaly_detectors.ipynb
jupyter nbconvert --to notebook --execute --inplace notebooks/05_localization_and_explainability.ipynb
jupyter nbconvert --to notebook --execute --inplace notebooks/07_training_benchmarking_and_calibration.ipynb
```

The notebooks read real MVTec data and committed metadata, show evidence inline, and do not write screenshot, CSV, JSON report, or chart files to external result folders. Heavy training is performed intentionally through scripts rather than rerun every time the presentation notebooks execute.

13. Risks, Limitations, and Mitigations

Risk	Impact	Mitigation
Missing heavy checkpoints	Most promoted advanced models cannot run from this branch alone	Distribute artifacts separately or export additional OpenVINO models; retain explicit baseline fallback
Single advanced deployment	Only cable has committed OpenVINO IR	Prioritize export/parity checks for the next most valuable categories
Pixel F1 variation	Exact masks may under-cover or over-cover defects	Tune pixel thresholds and evaluate IoU/Dice per category and subtype
Validation provenance	Metrics use labelled MVTec test folders	Describe as project validation and run factory-specific acceptance testing later
Category selection error	Wrong category uses wrong normal representation	Require category input and reject unsupported/mismatched records
Model confidence	Distance-based confidence is not a safety probability	Keep score and threshold visible and calibrate with production data
Lighting and pose shift	Factory conditions may alter anomaly maps	Control acquisition and collect line-specific normal data
Runtime latency	Patch memory and first model load can be costly	Use cached compiled OpenVINO runtime and warm service instances
Dataset license	MVTec data cannot be assumed commercially redistributable	Keep dataset outside source control and review license conditions

13.1 Engineering controls already implemented

- Registry validation rejects unsupported categories.
- OpenVINO requires a complete XML/BIN pair before runtime readiness is reported.
- Checkpoint-loading and prediction errors use a dedicated `PadimInferenceError`.
- Runtime outputs identify the actual engine and fallback state.
- Anomaly maps and masks remain available for manual QA review.
- Threshold provenance and benchmark results remain attached to model metadata.
- Weak pixel-level results and incomplete checkpoint packaging are explicitly documented.

14. Milestone Completion and Handover

14.1 Completed deliverables

Deliverable	Evidence	Status
Advanced model definitions	PaDiM and PatchCore through Anomalib	Complete
Category registry	15 promoted model selections and thresholds	Complete
Candidate benchmarking	Stored comparisons for cable, grid, hazelnut, and screw	Complete
Advanced evaluation	Image/pixel metrics for all 15 categories	Complete
Threshold calibration	14 category holdout records plus bottle legacy metrics	Complete
Advanced runtime	Cable PatchCore OpenVINO and spatial calibrator	Complete
Localization	Map, mask, heatmap, overlap, area, and critical-zone evidence	Complete
Fallback behavior	Explicit advanced-to-portable runtime path	Complete
Notebook evidence	Three executed Milestone 2 notebooks with inline output	Complete
Focused testing	14 passed and 1 controlled skip	Complete

14.2 Handover to Milestone 3

Milestone 2 hands forward a category-aware binary anomaly decision, detection confidence, anomaly score, localized mask, heatmap, area ratio, centroid, critical-zone status, engine identity, and fallback provenance. Milestone 3 adds defect subtype classification and severity/QA decision policy to this evidence. It must keep detection confidence separate from subtype confidence and must not invent a subtype when the classifier artifact is missing or uncertain.

14.3 Final outcome

> **Milestone conclusion:** VisionInspect AI now has a measured advanced anomaly-detection layer across all 15 MVTec categories, category-specific model promotion, calibrated decisions, explainable localization, one executable advanced OpenVINO path, and a transparent portable fallback. The remaining packaging and pixel-overlap limitations are quantified and carried into later milestones.

References

[1] T. Defard, A. Setkov, A. Loesch, and R. Audigier, 'PaDiM: a Patch Distribution Modeling Framework for Anomaly Detection and Localization,' ICPR Workshops, 2021. https://arxiv.org/abs/2011.08785

[2] K. Roth, L. Pemula, J. Zepeda, B. Scholkopf, T. Brox, and P. Gehler, 'Towards Total Recall in Industrial Anomaly Detection,' CVPR, 2022, pp. 14318-14328.
https://openaccess.thecvf.com/content/CVPR2022/html/Roth_Towards_Total_Recall_in_Industrial_Anomaly_Detection_CVPR_2022_paper.html

[3] Anomalib documentation, Engine, model training, prediction, post-processing, and OpenVINO export. https://anomalib.readthedocs.io/

[4] OpenVINO documentation, model preparation and Intermediate Representation. https://docs.openvino.ai/2025/documentation.html

[5] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, 'MVTec AD - A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection,' CVPR, 2019. https://www.mvtec.com/research-teaching/datasets/mvtec

[6] VisionInspect AI Member 4 source repository, branch Himanshu-parhi-ai/ml, files under ml/, models/, notebooks/, scripts/, and tests/.

[7] VisionInspect AI executed notebooks 03, 05, and 07 plus category model metadata.

Appendix A. Code and Artifact Inventory

Type	Path	Milestone 2 responsibility
Source	ml/anomalib_trainer.py	Build advanced datamodule, model, preprocessor, and engine
Source	ml/padim_detector.py	Anomalib/OpenVINO prediction and spatial calibration
Source	ml/model_registry.py	Category model selection and artifact readiness
Source	ml/inference.py	Advanced selection, fallback, and output contract
Source	ml/heatmap.py	Heatmaps, overlays, IoU, and Dice
Script	scripts/train_category_models.py	Train category models
Script	scripts/benchmark_category_models.py	Compare advanced candidates
Script	scripts/promote_category_benchmarks.py	Promote selected candidate
Script	scripts/calibrate_category_thresholds.py	Calibrate score/spatial decisions
Script	scripts/export_openvino.py	Export checkpoints to OpenVINO IR
Artifact	models/category_model_registry.json	Selected model and threshold per category
Artifact	models/categories/*/model_metadata.json	Metrics, calibration, and benchmark provenance
Artifact	models/exported/cable/weights/openvino/model.xml + model.bin	Executable cable advanced model
Artifact	models/categories/cable/openvino_calibrator.npz	Portable cable spatial calibrator
Notebook	notebooks/03_advanced_anomaly_detectors.ipynb	Registry, metrics, and advanced runtime evidence
Notebook	notebooks/05_localization_and_explainability.ipynb	Ground-truth localization evidence
Notebook	notebooks/07_training_benchmarking_and_calibration.ipynb	Scripts, benchmarks, and calibration evidence
Test	tests/test_heatmap.py	Heatmap and localization behavior
Test	tests/test_kfold_validation.py	Validation helpers
Test	tests/test_portable_runtime.py	Artifact and fallback portability
Test	tests/test_prediction.py	Advanced runtime and explainability contract

Appendix B. Evidence Index

Evidence	Source	What it proves
Model-selection diagram	Registry + metadata	12 PaDiM and 3 PatchCore selections
Advanced metric chart	Notebook 03	All-category image and pixel evaluation
Candidate benchmark chart/table	Notebook 07 metadata	Why selected candidates were promoted
Calibration diagram/table	Notebook 07 + script	Threshold development and holdout protocol
Cable confusion matrix	Cable metadata	Spatial-calibrator out-of-fold behavior
Advanced runtime panel	Notebook 03	OpenVINO score, anomaly map, and localized heatmap
Raw inference table	Notebook 03	Backend-ready advanced result fields
Localization panel	Notebook 05	Input, map, heatmap, predicted mask, and ground truth
Overlap/geometry table	Notebook 05	IoU, Dice, coverage, area, centroid, and critical zone
Focused test output	Pytest	14 passed and 1 controlled skip

End of Milestone 2 documentation.